

mir1の書き方

mir1の書き方の基本	2
move命令(コピー命令)	3
add命令(足し算命令)	3
sub命令(引き算命令)	3
mullo命令(掛け算命令)	4
mulhi命令(掛け算命令)	4
div命令(割り算命令)	5
mod命令(割り算命令)	5
rishift命令(右シフト命令)	5
input命令(インプット命令)	6
output命令(アウトプット命令)	6
label命令(ラベル)	7
jump命令(無条件ジャンプ命令)	7
if命令(条件分岐命令)	7
各条件の意味	7
func命令とreturn命令(関数)	8
call命令(関数呼び出し命令)	8

mir1の書き方の基本

csv形式で保存するため、

,

を使います

命令(addやifなど)と変数の間や、変数と変数の間などに使用します

add,r3,r1,r2のような感じです

mir1の数字は全て10進数です

変数はr1などのr付き数字で表現します

r1から使用できます

r0は何を書き込んでも、いつ読み出しても常に0です

即値は数字のみで表現します

1や10といった感じです

変数はr0からr255までですが、mir1から変換するmsa1が256行までしか対応していないので、変数は順番に使っていけば足ります(先にプログラムメモリが不足します)

変換後のmsa1が256行を超えるとCPUに書き込めないので、注意してください

(mir1の256行ではなく、msa1の256行です)

変換後は変換前より長くなるので、mir1の時点で200行とかだとほぼ確実に実行できません。

move命令(コピー命令)

変数、または即値をコピーする命令です

move,r2,r1

なら、r1をr2にコピーします

move,r2,1

なら、r2に即値の1を入れます

move,r2,r1

のうちのr2の場所は変数のみで、r1の場所だけ変数と即値が指定可能です

add命令(足し算命令)

変数同士、または変数と即値を足す命令です

add,r3,r1,r2

なら、r1とr2を足した結果をr3に保存します

add,r3,r1,2

なら、変数のr1と即値の2を足した結果をr3に保存します

add,r3,r1,r2

のうちのr1とr3の場所は変数のみで、r2の場所だけ変数と即値が指定可能です

sub命令(引き算命令)

変数同士、または変数と即値で引き算を行う命令です

sub,r3,r1,r2

なら、r1からr2を引いた結果をr3に保存します

sub,r3,r1,2

なら、変数のr1から即値の2を引いた結果をr3に保存します

sub,r3,r1,r2

のうちのr1とr3の場所は変数のみで、r2の場所だけ変数と即値が指定可能です

mullo命令(掛け算命令)

変数同士、または変数と即値をかける命令です
掛け算結果の下位8bitの結果を保存します

mullo,r3,r1,r2

なら、r1とr2をかけた結果をr3に保存します

mullo,r3,r1,2

なら、変数のr1と即値の2をかけた結果をr3に保存します

mullo,r3,r1,r2

のうちのr1とr3の場所の変数のみで、r2の場所だけ変数と即値が指定可能です

mulhi命令(掛け算命令)

変数同士、または変数と即値をかける命令です
掛け算結果の上位8bitの結果を保存します

mulhi,r3,r1,r2

なら、r1とr2をかけた結果をr3に保存します

mulhi,r3,r1,2

なら、変数のr1と即値の2をかけた結果をr3に保存します

mulhi,r3,r1,r2

のうちのr1とr3の場所の変数のみで、r2の場所だけ変数と即値が指定可能です

div命令(割り算命令)

変数同士、または変数と即値を割る命令です
割り算結果の商の結果を保存します

div,r3,r1,r2

なら、r1をr2で割った結果をr3に保存します

div,r3,r1,2

なら、変数のr1を即値の2で割った結果をr3に保存します

div,r3,r1,r2

のうちのr1とr3の場所の変数のみで、r2の場所だけ変数と即値が指定可能です

mod命令(割り算命令)

変数同士、または変数と即値を割る命令です
割り算結果の余りの結果を保存します

mod,r3,r1,r2

なら、r1をr2で割った余りをr3に保存します

mod,r3,r1,2

なら、変数のr1を即値の2で割った余りをr3に保存します

mod,r3,r1,r2

のうちのr1とr3の場所の変数のみで、r2の場所だけ変数と即値が指定可能です

rishift命令(右シフト命令)

変数の論理右シフトを行う命令です

rishift,r3,r1,2

なら、変数のr1を即値の2回右シフトした結果をr3に保存します

rishift,r3,r1,2

のうちのr1とr3の場所の変数のみで、2の場所は即値のみです

input命令(インプット命令)

外部からインプットして変数に保存する命令です

input,r1,0

なら、ポート番号0からインプットした値をr1に保存します

input,r1,0

うちのr1の場所は変数のみで、0の場所は即値のみです

ポートは0から15まであります

output命令(アウトプット命令)

外部に変数や即値をアウトプットする命令です

output,0,r1

なら、変数のr1をポート番号0からアウトプットします

output,0,1

なら、即値の1をポート番号0からアウトプットします

output,0,r1

うちの0の場所は即値のみで、r1の場所だけ変数と即値が指定可能です

ポートは0から15まであります

label命令(ラベル)

条件分岐などでジャンプする時に使用するラベルを定義します

label,0

なら、この場所をラベル番号0として定義します

条件分岐などでジャンプする時はアドレス指定ではなくラベル指定でジャンプします
指定されたラベルの次の命令から実行されていきます

jump命令(無条件ジャンプ命令)

指定したラベルへジャンプする命令です

jump,0

なら、ラベル番号0へジャンプします

if命令(条件分岐命令)

条件を満たしている時だけ指定したラベルへジャンプする命令です

if,条件,変数1,変数2,条件を満たした時にジャンプするラベル
のように書きます

変数2の所は即値も使用できます

各条件の意味

条件がeqの時は、変数1と変数2が同じ時にジャンプ

条件がneqの時は、変数1と変数2が同じではない時にジャンプ

条件がbiの時は、変数1が変数2以上の時にジャンプ

条件がnbiの時は、変数1より変数2の方が大きい時にジャンプ

条件がodの時は、変数1と変数2の和が奇数の時にジャンプ

条件がnodの時は、変数1と変数2の和が偶数の時にジャンプ

func命令とreturn命令(関数)

関数呼び出しでジャンプする関数を定義します

```
func,0
```

なら、ここからreturnまでの命令を関数番号0として定義します

関数呼び出しもアドレス指定ではなく、関数番号指定で行います

関数の終了を指定するreturn命令がないと、元のプログラムに戻れないので注意してください

```
func,0
```

```
add,r1,r1,1
```

```
return
```

なら、add,r1,r1,1を関数番号0として定義されます

全ての変数はグローバル変数で、ローカル変数はありません

プログラムは上から順番に実行されるので、一番上にメインプログラム、その下に関数を定義してください

メインプログラムの最後に何もしない無限ループなどがないと、関数のプログラムを実行し始めてしまうので注意してください

call命令(関数呼び出し命令)

指定した関数を呼び出す命令です

```
call,0
```

なら、関数番号0の関数を呼び出します